Genome analysis

RapidMACS: MACS3-identical peak calling, 50× faster

Ling-Hong Hung[1,*] and Ka Yee Yeung[1]

[1]School of Engineering and Technology, University of Washington Tacoma, Tacoma, WA, USA

[*]Corresponding author. lhhung@uw.edu

Abstract

Motivation: **MACS3 is a comprehensive peak-calling toolkit whose subcommands span single- and paired-end data, narrow and broad peaks, and a range of signal-track utilities. However, many ATAC-seq and multiomic pipelines, including our own, use just one of those capabilities: narrow peak calling. A single analysis may call peaks under different conditions, so a per-call saving is multiplied and the time recovered can be substantial. Furthermore, we wanted an efficient and embeddable narrow peak caller that could be integrated directly with our Chromap Suite aligner, so we wrote RapidMACS.**

Results: **RapidMACS is a narrow peak caller optimized for this purpose, building its signal tracks in a single lazy sweep that avoids a global sort, processing chromosomes in parallel, and keeping intermediates in memory rather than creating temporary files. In benchmarks involving single-cell ATAC-seq, bulk ATAC-seq, ChIP-seq, and CUT&RUN, it is 3.5–51× faster than MACS3 v3.0.3 depending on the applications and number of threads used. More importantly, the output is byte-identical. This means that any downstream analysis using RapidMACS will produce identical results to those using MACS3. The speed gains are largely due to these algorithmic changes rather than the choice of language, because MACS3's peak-calling code is itself compiled (Cython). RapidMACS depends only on htslib and zlib and links as a small static archive without a Python or Cython runtime.**

Availability and implementation: **RapidMACS is open source under the MIT license at `https://github.com/morphic-bio/rapidmacs`, with both a standalone CLI executable (`rapidmacs`) and a linkable C++ library. Prebuilt containers for x86-64 and arm64 are published as `biodepot/rapidmacs` and `ghcr.io/morphic-bio/rapidmacs`.**

Contact: **lhhung@uw.edu**

Supplementary information: **Supplementary data are available at *Bioinformatics* online.**

Keywords peak calling, MACS3, ATAC-seq, single-cell ATAC-seq, CUT&RUN, ChIP-seq, C++ library

1. Introduction

MACS (Zhang et al., 2008) has been the standard narrow peak caller for chromatin assays for nearly two decades, and it has remained so even as other platforms began including their own. For example, Cell Ranger ARC's ATAC peaks are wider, and downstream analyses routinely re-call narrow peaks with MACS through Signac (Stuart et al., 2021), ArchR (Granja et al., 2021), and SnapATAC2 (Zhang et al., 2024); ENCODE's ATAC-seq and transcription-factor peak sets are MACS calls (ENCODE Project Consortium, 2012). Narrow peaks are necessary for these analyses: accessibility and sequence-specific factor binding produce localized signals a few hundred bases wide. Narrow peak callers such as MACS3 return the summit, the single base of highest pileup within a peak, which is used to define a peak region. Motif enrichment and footprinting are anchored on summits—ArchR, for example, extends each summit by ±250 bp into the 501 bp fixed-width, non-overlapping peak set to create its cell-by-peak count matrix. Multiple MACS3 calls are often required. One cell contributes at most a fragment or two at any locus, so peaks can be called only after cells are aggregated to form a pileup. These pileups cannot be recovered from an earlier call's output, so every grouping means another MACS invocation: ArchR and SCENIC+ both call it once per group and merge the results (Bravo González-Blas et al., 2023).

Our contributions.

Our work in the MorPhiC consortium (Adli et al., 2025) required an efficient open-source peak caller that produced narrow peaks needed for downstream analyses. MACS3 is open source (BSD-3-Clause) and already fast, being implemented in compiled Cython against the CPython runtime. However, we had to invoke it separately, which meant writing intermediate files and reading them back. Linking MACS3 directly was not a feasible solution because Cython requires the Python interpreter. The alternative of invoking `macs3` as a subprocess would not have solved the problem of writing intermediate files. We therefore wrote RapidMACS so that it could

be linked directly into our Chromap Suite aligner, which extends Chromap (Zhang et al., 2021), and into the rest of our pipeline for ATAC-seq and multiomic analyses. Although we only needed to call narrow peaks in paired-end fragments from ATAC-seq, both ChIP-seq and CUT&RUN use essentially the same methodology, so we also added support for them. In addition to the linkable C++ library that we needed, we also added a CLI so that RapidMACS can also be used as a standalone program in any workflow that currently invokes `macs3 callpeak` to call narrow peaks.

2. Implementation

Scope and limitations.

RapidMACS calls narrow peaks, with or without a matched input or IgG control, from fragment, BAMPE, BEDPE, or single-end BED input. It does not support any other MACS3 subcommands; broad calling in particular is a separate algorithm, with its own pair of cutoffs and region-linking pass. Single-end input additionally requires `-nomodel`, since MACS3's fragment-size model building is not implemented. This focus allowed us to optimize RapidMACS extensively; the changes are described next.

Methodological changes.

MACS3's peak-calling path is compiled Cython rather than interpreted Python and the speed gains are not primarily due to the language differences but to three methodological changes. Two are structural: first, chromosomes are independent once the fragments are grouped, so RapidMACS calls them concurrently rather than in sequence, and second, it holds each chromosome's tracks in memory where MACS3 serializes them to a temporary file between passes. The third is how the pileups themselves are built, which is the central algorithm and the subject of the rest of this section.

To call a peak we need two tracks: the treatment track, holding the number of fragments covering each base, and the local background it is compared against. Both RapidMACS and MACS3 store these tracks as runs of constant value rather than as per-base arrays—what differs between the implementations is how those runs are built.

For single-cell ATAC-seq, the input is a fragment TSV file, from Cell Ranger or from Chromap. These are sorted by start position, but not by end position. MACS3 sorts the ends, then walks the sorted start and end arrays together to build the treatment track. It builds the background track the same way, from endpoints widened into a 10 kb window, which yields the average coverage over that window, with the genome-wide mean as a floor.

RapidMACS replaces that global sort with a lazy one that orders only the ends of the fragments spanning the current position. It keeps those pending ends in a heap and pops them as the sweep reaches them, which yields the same sequence of events a full sort would. A single sweep generates both the treatment and the background track, using a separate heap for each. This trades $O(n \log n)$ for $O(n \log k)$, and on the single-cell ATAC data the two are far apart: n is 53.0 M fragments, while k—the number spanning a given position—averages 4 for the treatment track and 510 for the 10 kb background window. Each heap is therefore a few kilobytes and stays in first-level cache for the whole sweep.

For bulk ATAC-seq, ChIP-seq, and CUT&RUN, coordinate-sorted BAMPE input is the norm, and sorting on the start coordinate very nearly sorts the ends along with it: both endpoints come from the same read pair, and the fragment between them is only a few hundred bases. Sorting an array already that close to order is cheap, so there is little for a lazy sweep to save. The heap method remains available through flags, but our preliminary tests showed it to be no faster, and slightly slower on ChIP-seq, because of the overhead of creating and managing the heap. RapidMACS therefore uses a single global sort on BAMPE by default. In all cases the two tracks are then overlaid, and each run over which both are constant is scored once by the Poisson tail probability of seeing that many fragments under the local background. A peak is then a stretch of runs clearing the threshold that is uninterrupted by a gap wider than a set width, and kept only if what results exceeds a minimum length.

Two ways to use it.

As a library, RapidMACS builds to a static archive depending only on htslib (Bonfield et al., 2021) and zlib, with no Python or Cython runtime. Its entry point, `RunMacs3FragPeakPipelineFromSortedIterator`, asks the caller for fragments one at a time, so they can come from a file or straight out of an array the host is already holding. As a standalone program, `rapidmacs` reads a Chromap/ARC fragments TSV, a single-end BED, or, via htslib, BAMPE or BEDPE, and writes narrowPeak and summit files in place of `macs3 callpeak`.

Open source made exact parity possible.

We wrote RapidMACS from MACS3's published description rather than porting its code. No MACS3 code is incorporated and RapidMACS is independent of MACS3's codebase. At every stage the development procedure was the same: run the new implementation and the reference MACS3 implementation on the same input and compare the output files. RapidMACS was written with AI-assisted programming under human direction, following the approach we describe for STAR Suite (Hung and Yeung, 2026b); the stage-by-stage output comparison against a reference implementation also verifies work produced by the AI agents.

MACS3's documentation leaves two things unclear: the tie-break it applies when several positions inside a peak share the maximum pileup, and the 32-bit arithmetic of its score tables. Both must be matched exactly for the output files to agree. Because MACS3 is open source we could read and instrument v3.0.3 to establish what it does in each case. Against a closed peak caller we could still have checked byte-identity, since that needs only the two output files. What we could not have determined was why they differed, and the likely outcome would have been a tool that agreed closely but not exactly.

3. Results

We validated on four full public datasets covering single-cell ATAC-seq, bulk ATAC-seq, ChIP-seq, and CUT&RUN: the 10x 3K PBMC Multiome ATAC channel (10x Genomics, 2021); ENCODE bulk ATAC-seq in GM12878 (ENCSR095QNB) (ENCODE Project Consortium, 2020); ENCODE CTCF ChIP-seq in NCI-H929 with its matched input control (ENCFF686KKV/ ENCFF181CXT) (ENCODE Project Consortium, 2021); and K562 CTCF CUT&RUN with matched IgG (SRR14255054/ SRR14255053) (Ng et al., 2023). The public sets were ingested directly from the fragments or sorted BAMPE files provided. As a further control, the two ATAC datasets were additionally realigned from the same reads with Chromap Suite, because that is how ATAC data reaches the peak caller in our MorPhiC workflows. Each ATAC

Table 1 Complete-program wall time against MACS3 v3.0.3 across six input configurations. Each row is one benchmark input, used either as its source distributes it or realigned by Chromap Suite from the same FASTQ files. Times are wall-clock seconds for the complete command-line program, medians of three recorded warm-cache runs after one unrecorded warm-up. All runs ran on a single server (i9-13900KF, 128 GB RAM) with every input, output, and temporary file on the same NVMe device. T is the number of RapidMACS threads. MACS3's peak-calling path is single-threaded so each row has only one MACS3 time. Parenthesized values are speedups against it. Peak counts differ between rows because the inputs differ. Within every row, and in every run, both programs produced byte-identical narrowPeak and summit files. Stage specific timings are in Supplementary Section S5.

	MACS3	RapidMACS			
Assay and input	**v3.0.3**	T=1	T=4	T=24	**Peaks**
scATAC-seq, 10× fragments	747.55	57.87 (12.9×)	24.13 (31.0×)	16.37 (45.7×)	50,003
scATAC-seq, Chromap fragments	758.62	58.88 (12.9×)	25.38 (29.9×)	16.12 (47.1×)	50,274
Bulk ATAC-seq, ENCODE BAM	146.63	41.49 (3.5×)	13.14 (11.2×)	7.28 (20.1×)	74,000
Bulk ATAC-seq, Chromap BAM	122.92	32.10 (3.8×)	10.74 (11.4×)	6.77 (18.2×)	78,574
ChIP-seq, ENCODE BAM + input control	616.90	96.39 (6.4×)	27.23 (22.7×)	12.11 (50.9×)	33,985
CUT&RUN, GEO BAM + IgG control	64.74	9.81 (6.6×)	2.98 (21.7×)	1.99 (32.5×)	9,097

dataset therefore appears twice in Table 1—the same reads, aligned once by its source and once by Chromap—which with ChIP-seq and CUT&RUN makes six configurations.

On all six, every narrowPeak and summit field is byte-identical to MACS3 v3.0.3—intervals, peak names, summit offsets and coordinates, signal values, p-scores and q-scores—and the full-file MD5 sums match. By identical, we mean that literally: the same bytes, not the same peaks within a tolerance. Anything computed from a RapidMACS peak or summit file—peak annotation, differential accessibility, motif enrichment, a cell-by-peak count matrix—returns exactly what it would have returned from the MACS3 file, because it is handed the same input file.

Table 1 gives the complete running time for every configuration. RapidMACS is 3.5–51× faster depending on assay, input, and thread count, and all 72 measured runs reproduced the byte-identical output described above. Single-cell ATAC-seq has a greater performance gain because it takes fragment inputs and can use the lazy heap based sweep to avoid an expensive global sort (12.9× faster on a single thread). The other assays take sorted BAMPE as input. Without the advantage of avoiding an expensive global sort, RapidMACS is only 3.5–6.6× faster on a single thread. Chromosome-level parallelism then multiplies both, by a further 3.5–8.0× at 24 threads. The BAMPE configurations scale better with threads because a BAM file is a series of independently inflatable BGZF blocks, so decompression itself parallelises. A plain gzip fragment file is one deflate stream whose decoder state carries from the first byte to the last, leaving no offset a second thread could start from, and RapidMACS inflates it serially. The higher throughput does come with a modest increase in memory usage. Peak resident set size is 0.46–5.44 GiB against 0.35–2.74 GiB for MACS3, roughly 1.2–2.6 times as much and at most about 2.7 GiB more.

One property of the released software affects the two assays with controls. MACS3 v3.0.3 writes each chromosome's control pileup to a fixed file name before returning it. As far as we can tell, nothing reads the file back. We found no option to disable it, so we built a version without those four lines. For ChIP-seq, whose control is the larger of the two at 39.3 M fragments, the writing takes 207.6 s, a third of MACS3's 616.90 s run. Both builds call identical peaks; only the writing differs. Against the build without it RapidMACS is 33.8× faster at 24 threads rather than 50.9×, which leaves single-cell ATAC-seq the fastest configuration at 47.1×. Supplementary Section S4 gives both in full. Treatment-only runs never reach that code. The complete 72-run matrix and the per-stage timings are given in Supplementary Sections S2–S5.

4. Applications

RapidMACS is provided as a libRapidMACS library as well as a standalone program because peak calling sits in the middle of pipelines rather than at the end of them. A pipeline that maps reads and then calls peaks by invoking `macs3` has to write its fragments to disk, start a Python process, and have that process read them back. For the single-cell ATAC dataset used here this involves transferring 2.3 GB of data going back and forth between processes via writing and reading temporary files. Linking to the library removes this overhead: an aligner then hands libRapidMACS the fragments it has just produced, in the same process directly through its C++ API. That is how we use it. Chromap Suite links RapidMACS to call ATAC peaks during the mapping pass, and Multiomics Suite composes that with transcriptomic processing to run alignment, peak calling, and cell calling for joint RNA and ATAC in one invocation of one binary (Hung and Yeung, 2026a).

The standalone program serves the more common use case: re-calling peaks that a pipeline has already produced. We verified RapidMACS against the peak-calling commands issued by the three tools that do it most: Signac, which calls MACS to replace Cell Ranger ARC's wider peaks on multiome data (Stuart et al., 2021); ArchR, which calls it once per pseudobulk replicate (Granja et al., 2021); and SCENIC+, which assembles a consensus enhancer set from per-cluster pseudobulk (Bravo González-Blas et al., 2023). All three use `macs2` by default but can use `macs3` if desired. We tested RapidMACS against the MACS3 v3.0.3 commands and confirmed that the narrowPeak and summit files are byte-identical.

5. Acknowledgments

We thank X. Shirley Liu's group, where MACS was originally developed, and Tao Liu and the MACS3 contributors, who have maintained and extended it since, for sustaining MACS over many years and for releasing it under a permissive open-source license: being able to read and run the reference implementation is what made byte-level parity possible. We also thank the htslib authors.

6. Funding

L.H.H. and K.Y.Y. at the University of Washington are supported by National Institutes of Health (NIH) grant U24HG012674. K.Y.Y. is also supported by the Virginia and Prentice Bloedel Endowment at the University of Washington.

7. Conflicts of interest

L.H.H. and K.Y.Y. have equity interest in Biodepot LLC. The terms of this arrangement have been reviewed and approved by the University of Washington in accordance with its policies governing outside work and financial conflicts of interest in research.

8. Data availability

Source, build instructions, and the parity harness are at `https://github.com/morphic-bio/rapidmacs`. All benchmark inputs are public; accessions and checksums are listed in Supplementary Section S1, and the complete 72-run benchmark matrix is given in Supplementary Section S3. The audit reproducing the Signac, ArchR, and SCENIC+ invocations, with the pinned upstream revisions and per-file checksums, is at `paper/WORKFLOW-COMPATIBILITY.md`.

References

10x Genomics. PBMCs from a healthy donor, 3k cells: Single cell multiome atac + gene expression. Public dataset, 2021. `https://www.10xgenomics.com/datasets/pbmc-from-a-healthy-donor-granulocytes-removed-through-cell-sorting-3-k-1-standard-2-0-0`.

M. Adli et al. MorPhiC consortium: towards functional characterization of all human genes. *Nature*, 638(8050):351–359, 2025. doi: 10.1038/s41586-024-08243-w.

J. K. Bonfield, J. Marshall, P. Danecek, H. Li, V. Ohan, A. Whitwham, T. Keane, and R. M. Davies. Htslib: C library for reading/writing high-throughput sequencing data. *GigaScience*, 10(2):giab007, 2021. doi: 10.1093/gigascience/giab007.

C. Bravo González-Blas, S. De Winter, G. Hulselmans, N. Hecker, I. Matetovici, V. Christiaens, S. Poovathingal, J. Wouters, S. Aibar, and S. Aerts. SCENIC+: single-cell multiomic inference of enhancers and gene regulatory networks. *Nature Methods*, 20(9):1355–1367, 2023. doi: 10.1038/s41592-023-01938-4.

ENCODE Project Consortium. An integrated encyclopedia of DNA elements in the human genome. *Nature*, 489(7414):57–74, 2012. doi: 10.1038/nature11247.

ENCODE Project Consortium. ATAC-seq on GM12878. ENCODE experiment ENCSR095QNB, 2020.

ENCODE Project Consortium. CTCF ChIP-seq in NCI-H929 with matched input. ENCODE experiments ENCSR634OAQ and ENCSR409FGC, 2021.

J. M. Granja, M. R. Corces, S. E. Pierce, S. T. Bagdatli, H. Choudhry, H. Y. Chang, and W. J. Greenleaf. Archr is a scalable software package for integrative single-cell chromatin accessibility analysis. *Nature Genetics*, 53(3):403–411, 2021. doi: 10.1038/s41588-021-00790-6.

L.-H. Hung and K. Y. Yeung. Multiomics suite: a single open-source binary for joint single-cell multi-modal profiling. *Manuscript in preparation*, 2026a.

L.-H. Hung and K. Y. Yeung. Star suite: an open-source single-binary platform for agentic transcriptomics, built for the nih morphic consortium with ai-assisted programming, 2026b. URL `https://doi.org/10.64898/2026.03.09.710580`. Preprint.

M. Ng, L. Verboon, H. Issa, R. Bhayadia, M. W. Vermunt, R. Winkler, L. Schüler, O. Alejo, K. Schuschel, E. Regenyi, D. Borchert, M. Heuser, D. Reinhardt, M.-L. Yaspo, D. Heckl, and J.-H. Klusmann. Myeloid leukemia vulnerabilities embedded in long noncoding rna locus MYNRL15. *iScience*, 26:107844, 2023. doi: 10.1016/j.isci.2023.107844.

T. Stuart, A. Srivastava, S. Madad, C. A. Lareau, and R. Satija. Single-cell chromatin state analysis with signac. *Nature Methods*, 18(11):1333–1341, 2021. doi: 10.1038/s41592-021-01282-5.

H. Zhang, L. Song, X. Wang, H. Cheng, C. Wang, C. A. Meyer, T. Liu, M. Tang, S. Aluru, F. Yue, X. S. Liu, and H. Li. Fast alignment and preprocessing of chromatin profiles with chromap. *Nature Communications*, 12(1):6566, 2021. doi: 10.1038/s41467-021-26865-w.

K. Zhang, N. R. Zemke, E. J. Armand, and B. Ren. Snapatac2: a fast, scalable and versatile tool for analysis of single-cell omics data. *Nature Methods*, 21:1742–1751, 2024. doi: 10.1038/s41592-024-02322-5.

Y. Zhang, T. Liu, C. A. Meyer, J. Eeckhoute, D. S. Johnson, B. E. Bernstein, C. Nusbaum, R. M. Myers, M. Brown, W. Li, and X. S. Liu. Model-based analysis of chip-seq (macs). *Genome Biology*, 9:R137, 2008. doi: 10.1186/gb-2008-9-9-r137.